

fid-inc-smoke____fidelity-kary-f-k4-inc

An AgentMPI run analysis

Abstract

This is the smallest run in my set — 43 events, 0.032 s, three operator applications — and it is a smoke test of the `-incompressible` payload generator and the new value-level retention metrics ahead of the broker campaign that ran ten minutes later. The executor is `function` $\times$ 8, so it measures the protocol and the k -ary algorithm’s arithmetic, not agent behaviour. On that arithmetic it is exactly right: `fold_depth`: $2 = \lceil \log_4 8 \rceil$ against the binomial sibling’s 3 and the chain’s 7, three applications $= \lceil (p-1)/(k-1) \rceil = \lceil 7/3 \rceil$ recorded as two `merge:k4:d1` calls plus one `merge:k2:d2`, seven messages $= p-1$, and `variadic`: `true` confirming the four-way kernel actually ran rather than degrading to a left fold. Retention is 1.000 with `n_intact`: 96 and `n_dropped`: 0, and that is a plumbing result rather than a fidelity one: the surrogate kernel never sees the token budget, so its outputs are 780, 780 and 2,043 tokens against a nominal ceiling of 450, and the recovered accumulator for the $4 \rightarrow 0$ message carries all five candidate items — four whole input reports plus the prompt’s own instruction line — so nothing was discarded at all. The run does, however, expose a real defect in the cost model, and it is the one my other k -ary documents point back to: the collectives table reports 2 rounds against a prediction of 1, because `cost.py` evaluates the tabulated ("`reduce`", "`kary`") entry at `DEFAULT_FANIN = 8` rather than at the fan-in the run used, and the run-level finding still announces that every checkable collective agrees with its cost expression, because `Analysis.model_checks` aggregates only `messages_agree`.

1 What this run is

property	value
run	fid-inc-smoke____fidelity-kary-f-k4-inc
experiment	-
campaign label	-
declared world size	8
ranks appearing in the log	8
executors	<code>function</code> $\times$ 8
events	43
wall time	0.03 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.06×	total busy time over wall time
parallel efficiency	0.8%	achieved parallelism per rank
idle fraction	99.2%	rank-seconds paid for and unused
peak ranks busy	1	of 8 in the world
serial fraction of active time	100.0%	time with exactly one rank busy
load imbalance	4.00×	slowest rank’s busy time over the mean
coordination share	22.3%	rank-seconds blocked in collectives, of those available
coordination in flight	83.0%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	3	invocations that reached a model
agent latency p50 / p95	0.00 / 0.00 s	per invocation
messages	7	7 eager, 0 rendezvous
tokens sent	1,868	0 deferred under rendezvous
tokens in / out	3,233 / 3,603	prompt and completion
cost	\$0.0637	0.0177 \$/kTok out

3 What the analysis notices

- [\[note\]](#) Occupancy is not measurable for this run: its executors (function) emit no broker claim events, so busy time, achieved parallelism, and the idle fraction are artefacts of the instrumentation rather than properties of the run.
- [\[note\]](#) Too few distinct output sizes to fit β , so the reported latency parameters are medians rather than a regression.
- [\[note\]](#) Load imbalance is 4.00×: the slowest rank was busy far longer than the mean.
- [\[ok\]](#) All 1 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.
- [\[ok\]](#) All 1 checkable collective(s) also matched the predicted round count.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.00	3.7	1	2	0	4	0	0.0	finalized
1	0.00	0.0	0	0	1	0	178	0.0	finalized
2	0.00	0.0	0	0	1	0	178	0.0	finalized
3	0.00	0.0	0	0	1	0	178	0.0	finalized
4	0.00	6.5	1	1	1	3	800	0.0	finalized
5	0.00	0.0	0	0	1	0	178	0.0	finalized
6	0.00	0.0	0	0	1	0	178	0.0	finalized
7	0.00	0.0	0	0	1	0	178	0.0	finalized

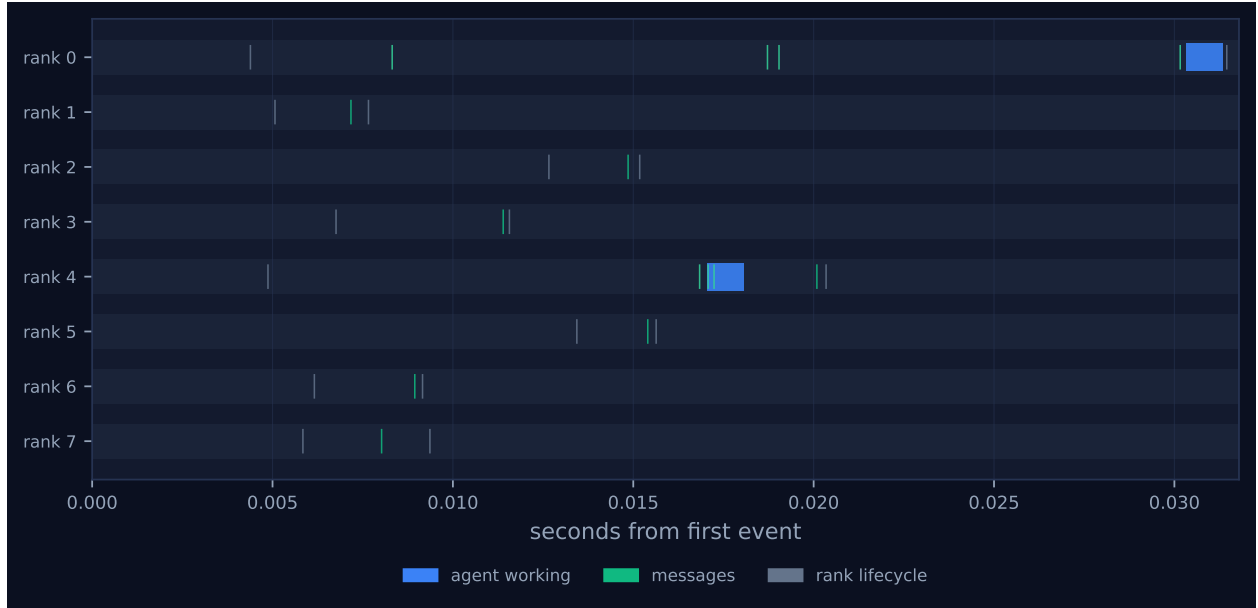


Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer’s palette so the same shapes read the same way in both.

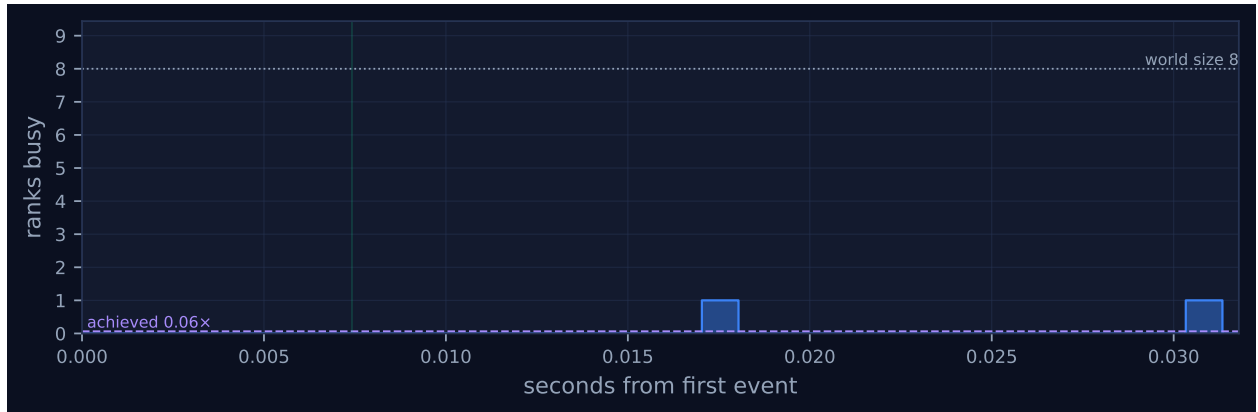


Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

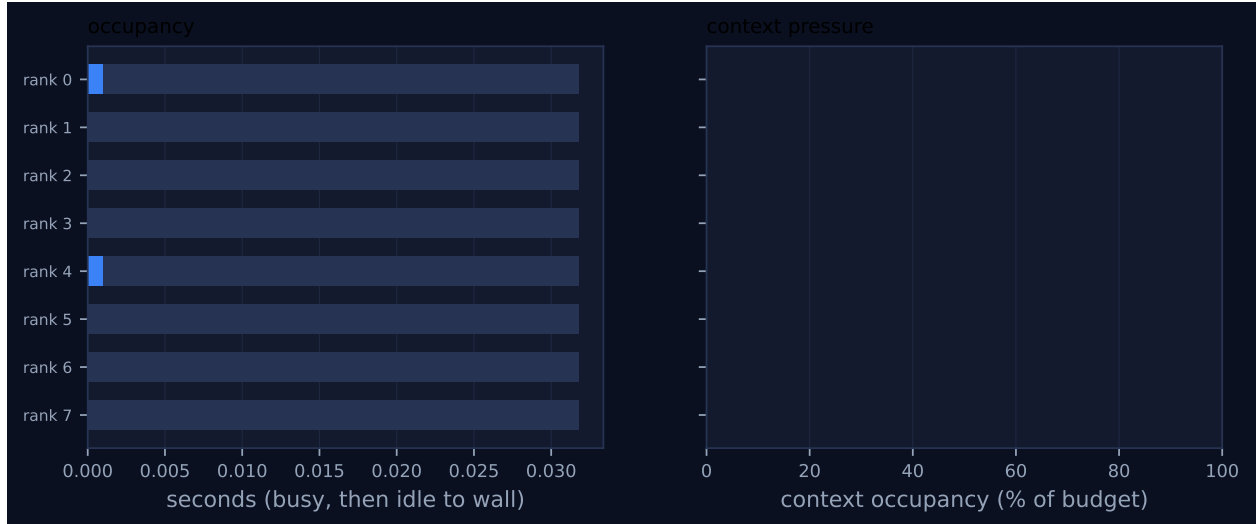


Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

Per-rank behaviour. Occupancy is busy time over the rank’s own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
reduce	kary	–	8	8	2	2	7	7	1,728	0.03	0.02	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 is the sparsest picture in my set: six ranks emit a single message tick inside the first 16 ms and exit, rank 4 folds four values at $t = 0.018$ s, rank 0 folds four at 0.020 s, and rank 0 folds the two accumulators at 0.031 s. That is the two-level k -ary tree drawn literally — at `stride = 1` the ranks with $vr \bmod 4 = 0$ each collect three children and combine all four in one application; at `stride = 4` rank 4 sends to rank 0, which combines two. Peak ranks busy is 2 of 8, which is not inefficiency but the arithmetic of a tree whose second level has one node, and the $4.00\times$ imbalance flagged in the findings is that concentration. The occupancy figures should be ignored: these executors emit no broker claim events and the generated findings say so.

The communication matrix, Figure 4, is two columns — $1, 2, 3 \rightarrow 0$ and $5, 6, 7 \rightarrow 4$ at 178 tokens each, then $4 \rightarrow 0$ at 800 — for seven messages and 1,868 tokens. That is the same message count as the

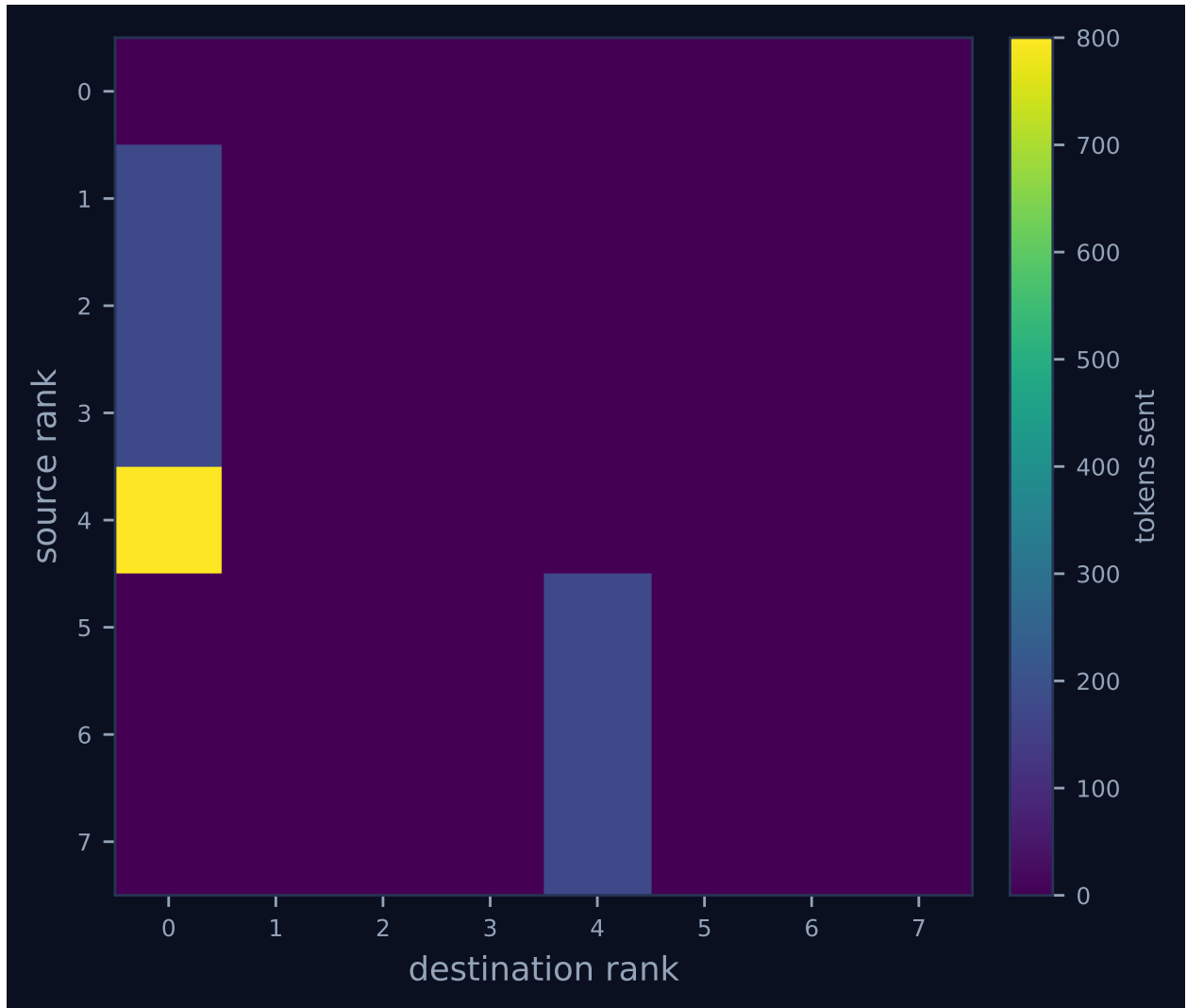


Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm’s shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

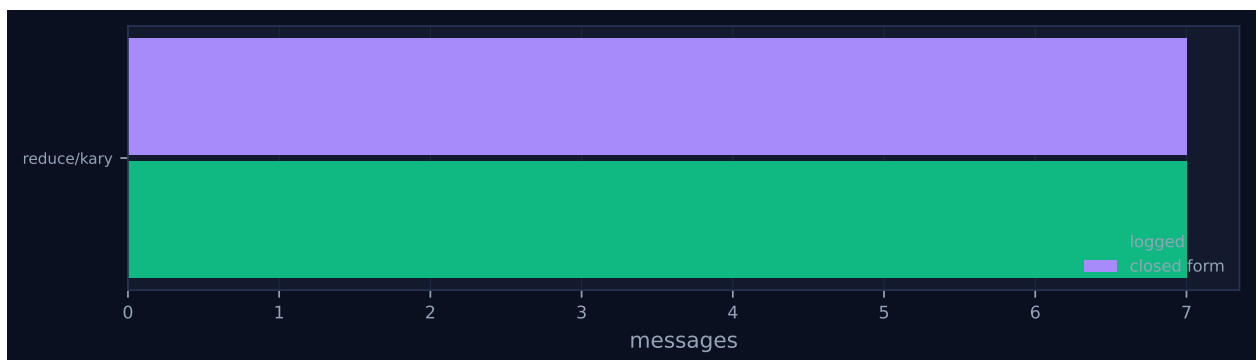


Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

binomial and chain siblings at this p , because every non-root rank sends exactly once whatever the fan-in, and 87% less traffic than the chain’s 14,950, because only one accumulator ever crosses the fabric instead of six.

7 Interpretation

The counts are by construction and worth checking explicitly, because `kary` appears in no collective sweep and these fidelity cells are its only sweep-adjacent evidence. Fold depth: the `kary` branch advances `stride` by factors of k and increments depth once per level, giving $\lceil \log_4 8 \rceil = 2$, and the `coll.reduce` payload records 2. Applications: each variadic call consumes k inputs and retires $k - 1$, so a p -rank fold needs $\lceil (p - 1)/(k - 1) \rceil = 3$ at $p = 8$ with $k = 4$, and the trace holds exactly three. The labels carry the actual arity, so the degenerate two-way top level — p is not a power of k here — is visible as `merge:k2:d2` rather than having to be inferred; a clean $k = 4$ measurement needs $p = 16$, which is what `fid-synth__fidelity-kary-f-k4` provides. Messages: seven, matching $p - 1$.

Retention 1.000 is not a fidelity result, for the same two reasons that hold across this smoke test. The budget is not binding: at commit 90ca469 the harness passed the module-level `MERGE_CONTRACT`, which has no `max_tokens` field, so the 450 tokens existed only as a sentence in the prompt and a `meta` hint — the advisory regime — which is why a 2,043-token output records `attempt: 1` and the run reports zero contract violations. And the operator is not budget-aware: `_surrogate_merge` takes a drop rate and a seed and grows without limit rather than choosing what to shed. Whole-rank erasure in this experiment is what happens when an accumulator standing for many ranks fills a binding budget, so with neither condition met the mechanism cannot fire, and `-incompressible` — whose purpose is to remove the operator’s ability to re-encode its way out of pressure — is inert. The recovered accumulator for the $4 \rightarrow 0$ message makes this concrete: five findings, 49 identifiers, ranks 4 through 7 complete plus the phantom [F-3-2] copied out of the prompt’s own instruction line, which is to say the kernel discarded nothing in its first application and its output is simply its four inputs quoted inside one another. The comparison this clears the way for is `inc-mb-fidelity-inc__fidelity-kary-f-k4-inc`: same algorithm, same p , same twelve incompressible items per rank, same nominal 450 tokens, but with `max_tokens: 450` in the contract and real agents behind it, and there the three applications return 449, 437 and 437 tokens, retention is 0.3854, and ranks 4 through 7 are erased whole.

The defect this run exposes is in the cost model rather than the algorithm, and it matters because it is the only place the k -ary implementation is checked against a closed form. The collectives table shows rounds 2 against a prediction of 1 with `rounds_agree: false` in `metrics.json`, and yet the model column reads ✓ and the run-level finding states that all checkable collectives sent exactly the number of messages their cost expression predicts. Both halves are wrong in different ways. The prediction is for a different experiment: the tabulated ("`reduce`", "`kary`") entry in `cost.py` evaluates $\lceil \log_k p \rceil$ at `DEFAULT_FANIN = 8`, not at the fan-in the run used, so it predicts the single-application depth-1 tree while the run performed the $k = 4$ two-level one. `predict_kary(p, n, k)` takes an explicit k and exists precisely for this, but `analysis.py` consults `FORMULAS`. And the reassurance is uninformative: `Analysis.model_checks` counts only `messages_agree`, so a round-count disagreement is recorded in the table and in `metrics.json` and never reaches the findings list. The same mismatch is present in the `sat-mb` and `inc-mb` k -ary cells, and is hidden by coincidence in the $p = 16$ one, where $\lceil \log_8 16 \rceil$ and $\lceil \log_4 16 \rceil$ are both 2.

8 Threats to this reading

The executor is a surrogate, so nothing here is agent behaviour, and the \$0.0637 is a modelled price for tokens produced by concatenation. Three applications on one trial is also the smallest sample in my set; no distributional claim should be made from it.

The kernel's loss model, and hence the meaning of retention 1.000, rests on reading the source and on the message payloads rather than on logged calls, because a **function** executor writes no rows to **agent_calls**: this run's fabric contains no prompts and no results. What is directly observed is that the $4 \rightarrow 0$ payload has five findings and 49 identifiers with the instruction sentence present, and that the two depth-1 applications had identical prompt sizes (773 tokens) and identical output sizes (780). That equality is worth one caution: the kernel seeds its generator from **seed + len(prompt)**, so structurally identical applications make identical draws — in the binomial sibling four prompts of 1,695 characters all dropped the same candidate, and in the $p = 16$ k -ary cell two identical prompts erased ranks 0 and 4 together. Here the identical draw happened to keep everything, so the defect is invisible in the result even though the mechanism is present.

Finally, this run's **value_retention: 1.0** and **n_label_only: 0** are the first exercise of the value-level metric block, and a smoke test returning the trivial answer does not show it can return a non-trivial one. That demonstration is **inc-mb-fidelity-inc**, where **n_dropped: 59** with **n_label_only: 0** establishes that the enforced-budget failure is clean deletion rather than a label surviving without its payload — the distinction the metric was added to draw.